

Message Queue

A **message queue** is a durable system component used for **asynchronous communication** between services.

It works as a **buffer** between producers and consumers.

Instead of direct service-to-service communication:

None

Service A → Service B

Use queue:

None

Service A → Message Queue → Service B

This improves scalability, reliability, and decoupling.

1. Core Components

Producer / Publisher

Creates messages and sends them to queue.

Examples:

None

Web Server

Payment Service

Order Service

Notification Service

Message Queue

Stores messages until processed.

Examples:

None

Kafka

RabbitMQ

Amazon SQS

Redis Streams

Consumer / Subscriber

Reads messages and processes them.

Examples:

None

Email Worker

Image Processor

Billing Worker

Analytics Service

2. Basic Architecture



None

Producer A └─

Producer B ┤───> Message Queue ───> Consumer 1

Producer C └─ Consumer 2

Consumer 3

Explanation

Multiple services generate work.

Queue stores tasks.

Consumers pick tasks one by one.

This prevents overloading downstream systems.

3. Why Message Queue Is Powerful

Without queue:

None

Web Server waits for worker response

With queue:

None

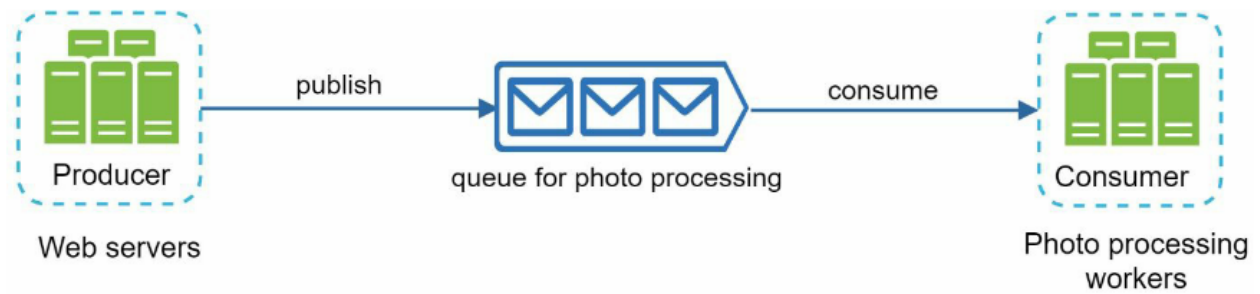
Web Server sends task instantly

Returns response immediately

Worker processes later

Better user experience.

4. Real Example



Suppose app supports:

None

Crop image

Sharpen image

Blur image

Resize image

Apply filters

These operations take time.

Flow

None

User uploads photo



Web Server creates job



Publishes message to queue



Photo Worker picks job



Processes image



Stores final result

5. Example Message

None

```
{  
  user_id: 55,  
  image_id: 902,  
  action: "blur",  
  intensity: 4  
}
```

Worker reads this and executes task.

6. Independent Scaling

If Queue Size Increases

None

10,000 pending jobs

Add more workers:

None

5 workers → 20 workers

If Queue Empty

Reduce workers:

None

20 workers → 3 workers

Saves cost.

7. Decoupling Benefit

Producer does not care:

None

Who processes job

When processed

How processed

Consumer does not care:

None

Which server created job

Both systems evolve independently.

8. Reliability Benefit

If worker crashes:

None

Message remains in queue

Another worker retries

If producer temporarily down:

None

Consumers still process existing messages

9. Common Use Cases

None

Email sending

SMS notifications

Image/video processing

Order processing

Payment retries

Analytics pipelines

Log processing

Background reports

10. FIFO vs Parallel Queues

FIFO Queue

Messages processed in order.

Useful for:

None

Financial transactions

Inventory updates

Standard Queue

Higher throughput, parallel workers.

Useful for:

None

Emails

Image jobs

Notifications

11. Queue Metrics to Monitor

None

Queue length

Oldest message age

Worker throughput

Failure rate

Retry count

Processing latency

12. Risks / Challenges

A. Duplicate Processing

Need idempotent consumers.

B. Poison Messages

Bad message fails repeatedly.

Use:

None

Dead Letter Queue (DLQ)

C. Backlog Growth

Need auto-scaling workers.

13. HLD Design Example

None

Client



Load Balancer



Web Servers



Message Queue



Worker Cluster



Database / Storage

14. Interview Answer

If asked how to process heavy background tasks:

None

Use message queue between web servers and workers.

Web server pushes jobs asynchronously.

Workers consume and process tasks.

Scale workers based on queue size.

Excellent scalable answer.

15. One-Line Summary

A message queue buffers asynchronous work between producers and consumers, enabling decoupling, fault tolerance, and independent scaling of background processing systems.